LONDON METROPOLITAN UNIVERSITY

Islington college
(इस्लिङ्टन कलेज)

CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework

2024/25 Spring

Student Name: Aasanna Khatiwada

London Met ID: 24046332

College ID: NP01NT4A240048

Assignment Due Date: Wednesday, May 14, 2025

Assignment Submission Date: Wednesday, May 14, 2025

Word Count: 3,938

Project File Links:

Onedrive Drive Link:	Keep Google Drive URL of your Project Here with Anyone in Organization can View Option Enabled

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

24046332 Aasanna Khatiwada.docx

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:95806914

Submission Date
May 14, 2025, 12:26 PM GMT+5:45

Download Date
May 14, 2025, 12:27 PM GMT+5:45

File Name
24046332 Aasanna Khatiwada.docx

File Size
27.7 KB

36 Pages

3,907 Words

20,404 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **20** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **8** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
7% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **20** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **8** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	islingtoncollege on 2025-05-13 — 1%
2	Submitted works	Nazarbayev University on 2024-09-05 — <1%
3	Submitted works	Manipal University Jaipur Online on 2025-01-02 — <1%
4	Submitted works	University of the West Indies on 2018-04-23 — <1%
5	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2023-09-03 — <1%
6	Submitted works	University of Warwick on 2022-06-30 — <1%
7	Internet	www.q7basic.org — <1%
8	Submitted works	Napier University on 2008-12-18 — <1%
9	Submitted works	Colorado Technical University Online on 2017-04-17 — <1%
10	Submitted works	University of North Texas on 2024-02-06 — <1%

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1 Introduction to Project

The project is a python project that consists of simple file handling and data presentation code that reads the information from a .txt file called “Product.txt” which has sunscreen cream from three different companies and three different countries. It uses python dictionary and lists to store and organize the data. The code processes the data and displays it in a readable format, to be precise a tabular format in the IDLE console. The txt file has three lines with 5 different columns as product name, company name, quantity, price and country of origin separated by comma. The python code performs two tasks. First, it reads all the lines from the Product.txt file and stores it in a dictionary. Item Id is used as a unique key which helps us organizing and referencing the products efficiently. Secondly, the code reads the file and prints the product data in a readable format. It includes 5 different columns separated by using escape sequence ‘\t’. The header has symbol number, Product name, Company name, quantity, price and country. It is than separated by printing dashes ‘-‘ below the header for neat presentation. The code not only reads and displays the content in the .txt file but is also able to edit the number of products in stock. It allows us to buy, sell or exit the interface in the console. The code also has many exceptions. The project also requires us add a condition where if a customer buys three or more than three products, a free product is also given. An exception is also added in the code where a customer buys high number of product than the quantity of stock.

The simple file handling code done in this project is also applicable and used in many real-world scenarios, usually in business and inventory management systems. The data is stored in plain text for ease of access and portability. The python project also depicts the same scenario of a sunscreen company called WeCare. This project is a good foundation for us beginners learning python programming who wants to learn how real-world data is collected, structured and displayed by using basic programming concepts.

1.2 Concepts Used

- **File handling**

  The project uses a concept called file handling which is extremely important and used widely to interact which external files. This is a critical concept used for dealing with real-world data stored outside the program. Some keywords used in file handling are: open(), readlines(), close() etc. The open() method is used to access and open the file outside the program, readlines() method is used to read all the lines in the file and return them as a list of strings where each string is a line from the file, close() method is used to close the file out the program after working on it.

- **Data Structures**

  A concept of dictionary is used in this project to store the products entries using numbers as keys which allows for easy reference and extensibility. The syntax for calling a dictionary is _variable_ = {key: value}.

- **String Manipulation**

  It is used to create, modify and analyse strings in python programming using built-in methods and operation. Once a string in created it can't be changed but can be modified and new string can be created with reference to it. Methods like .replace() and .split() are used in this project.

- **Loops**

  For loop is used in this project to iterate through the lines in the txt file for storing and displaying the data. It is also used to assign symbol number to the product while displaying.

- **Formatted Display**

  Escape sequence, tab and spacing are used in this project to display the information in the txt file in a clear and readable format.

1.3 Aims and Objectives

The aim of this project is to develop a python program which reads, processes and displays data in an organized and readable format from an external txt file. It also aims to introduce basic file handling concepts and demonstrate how real-world data can be used and managed using python programming. Moreover, it also aims to provide a foundation for developing more complex data management systems such as adding, deleting, updating, searching etc.

Objectives

i. Demonstrate file handling concept, including opening, reading and closing file outside the program properly.
ii. Store the data in a structured format using dictionary and displaying it in a readable format.
iii. To use loops and string manipulation properly to iterate each line in the txt file.
iv. To display the output in a clean and readable format using spacing and escape sequence method.
v. To provide a foundation for developing complex file handling systems.

1.4 Technologies Used

- Python Programming Language

  Python is a programming language which is used in various real-world technologies like web development, system scripting, software development etc. It works on different platform like windows, mac, Linux and many more. IT has a simple syntax and allows developers to code with fewer lines.

- Text File

  Text file is a file which uses .txt format to store data in it. It is used in this course work to store the data i.e. the attributes of products that are sold in the we care shop.

- Built in python Libraries and functions

  They are the default libraries and functions which are a set of modules that are available with python installation. They provide a wide range of functionality and operations. In this project built in functions like open(), print(), try-except for error handling were used to perform task for reading files and user input validation.

- IDLE

  IDLE is a default development environment for python programming. It is used to write, edit and test the scripts in a simple and beginner friendly interface.

- Draw.io for flowchart

  It is an application which is used to create flowchart of the program which helped to plan and understand the logic and flow of the program before writing the code.

- IDLE terminal

  It is a terminal which is used to compile and run the program and interact with it. It allows the user to input data, view outputs and perform various operations that the program allows.

2. Algorithm

Algorithm in programming is a set of defined instruction which are designed to perform a specific function. An algorithm can range from a simple process to complex series of operations which are used in machine learning. Algorithm of a program contains stages like input, processing, looping, decision, output, exit etc which are used to complete an algorithm which is then used to develop the program. The algorithm for the course are as follows:

Step 1: Start the program which displays a welcome message and company details.

Step 2: Load the product data from a txt file to a python dictionary

Step 2.1: Open the file with .txt extension

Step 2.2: Read reach line and split it into a list of product attributes like name, brand, price etc

Step 2.3: Store each product in a dictionary with Id as key

Step 2.4: If the file is not found, print a suitable error message and exit

Step 3: Show a menu with options

Step 3.1: Show three options which are in loop until option for exit is chosen

Step 3.2: Option 1 to sell product

Step 3.3: Option 2 to buy from manufacturer

Step 3.4: Option 3 to exit the system

Step 4: Take input from the user

Step 4.1: if option == 1, go to step 5 elif go to step 3

Step 4.2: If option == 2, go to step elif go to step 3

Step 4.3 If option == 3, go to step elif go to step 3

Step5: If option == 1 go to step 6 elif go to step 4

Step 6: Get the name and phone number of the customer as customer details and store it in suitable variables

Step 7: Ask for a product id to sell, if product id is valid go to step 7 else display a suitable message and go to step 6

Step 8: Get the quantity of product and validate the available stock

Step 5.1: If stock is validated apply the buy 3 get 1 free logic else go to step 5.2

Step 9: Deduct the quantity of stock for the product bought and calculate total

Step 8.1: Appen the products into a bill list.

Step 10: Ask if more products are to be sold.

Step 11: Calculate shipping cost if the total > 5000

Step 12: Display the bill and go to step 3

Step 13: If option==2 to step 13

Step 14: Display the table containing the product attributes in a user-friendly form

Step 15: Ask the user for product id that they want to restock

Step 16: If product id is valid ask for the quantity of product to restock elif go to step 14

Step 17: The amount of product to restock is added to current quantity of product in the display

Step 18: Ask if more products are to be restocked if not go to step 4

Step 19: If option == 3 go to step 20 elif go to step 4

Step 20: Print a suitable message to exit the system

Step 21: Close the .txt file and exit

3. Flowchart

A Flowchart is a pictorial representation of a program that visually defines the workflow, sequence and decisions that are involved in a program.

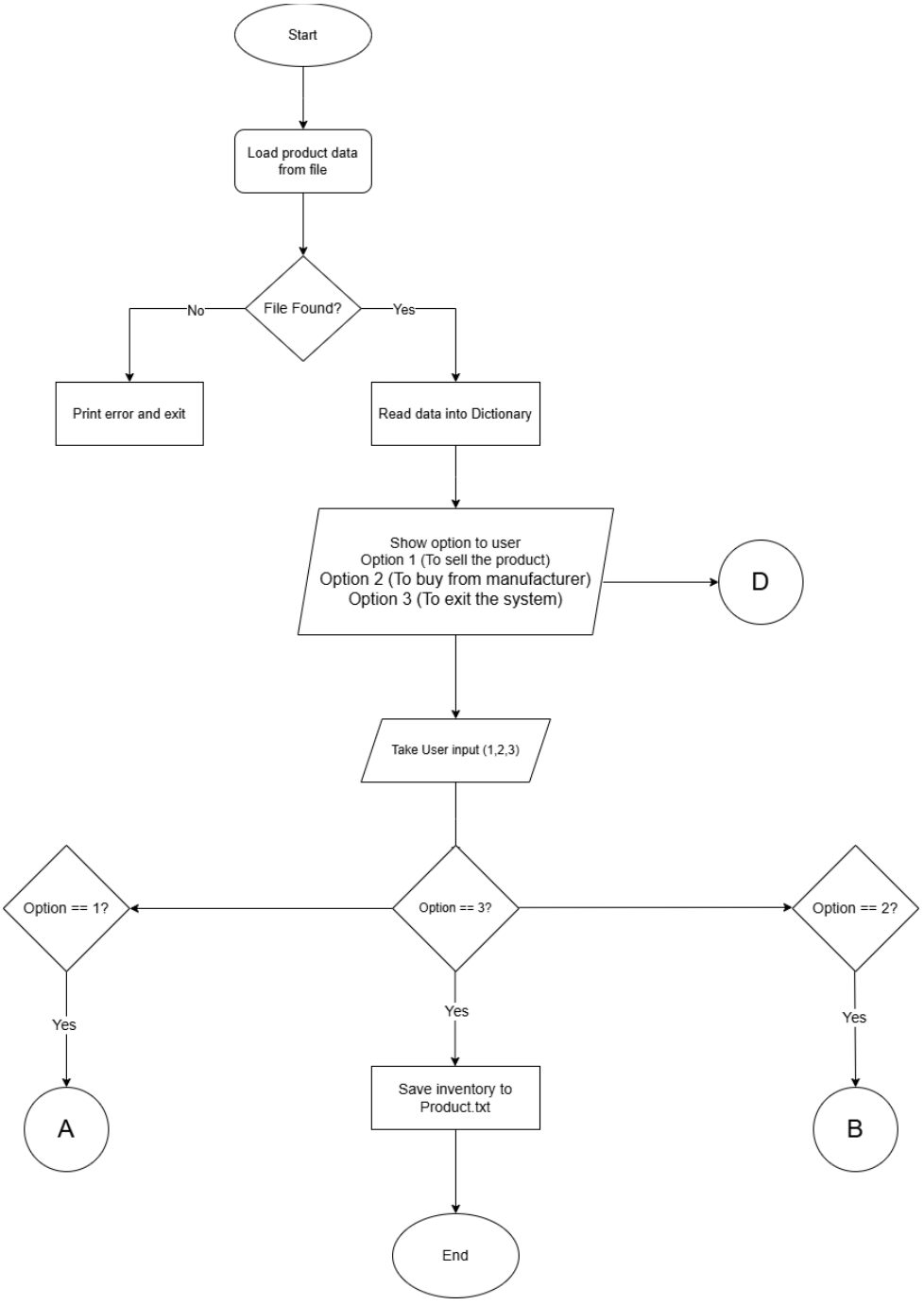

Figure 1: Flowchart part 1

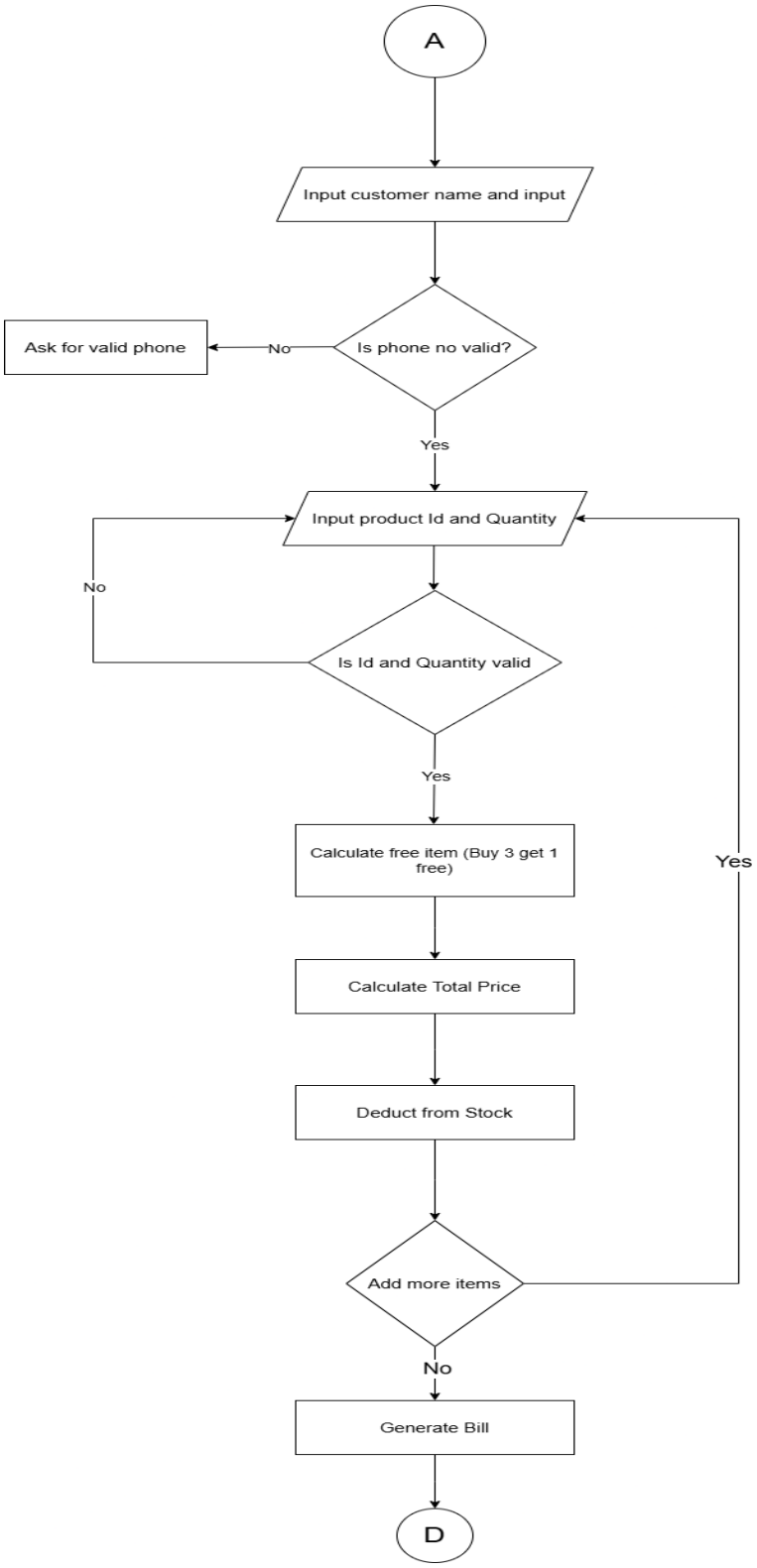

Figure 2: Flowchart part 2

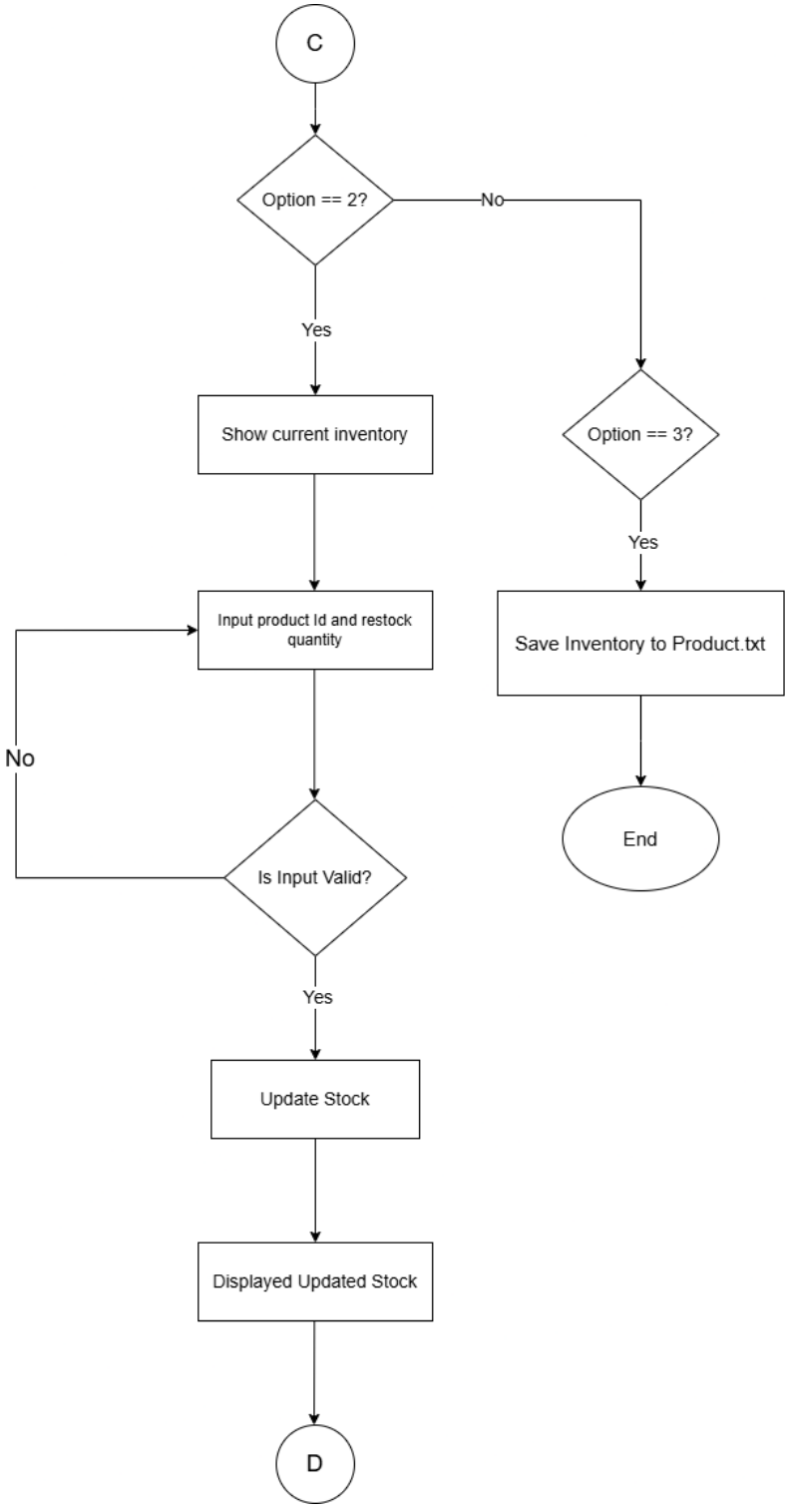

Figure 3: Flowchart part 3

4. Pseudo Code

4.1 Main module

BEGIN PROGRAM

IMPORT 'read_products' FROM 'read' MODULE

IMPORT 'save_sold_bill' AND 'save_restocking_bill' FROM 'write' MODULE

IMPORT 'sell_product', 'restock_product', 'display_products' FROM 'operation' MODULE

PRINT blank lines

PRINT header for WeCare Wholesale

PRINT location and contact details

PRINT welcome line

PRINT horizontal separator

CALL read_products() FUNCTION FROM 'read' MODULE

STORE returned product data IN 'products' VARIABLE

SET main_loop = TRUE

WHILE main_loop IS TRUE DO:

PRINT separator line

PRINT options menu for the admin:

1. Sell the product

2. Buy from Manufacturer

3. Exit the system

TRY:

PROMPT user to input an option (number)

CONVERT input TO integer AND STORE IN 'option'

EXCEPT ValueError:

PRINT invalid input message

CONTINUE loop to display options again

IF option == 1 THEN:

PRINT sell bill header

CALL sell_product(products) FUNCTION FROM 'operation' MODULE

STORE returned result IN 'result'

IF result IS NOT None THEN:

UNPACK result INTO:

- name
- phone number
- sold_items
- total

- shipping_cost

- grand_total

PROMPT user to confirm saving bill to text file (y/n)

IF user_input IS 'y' THEN:

CALL save_sold_bill(name, phone number, sold_items, total, shipping_cost, grand_total) FUNCTION FROM 'write' MODULE

ELSE IF option == 2 THEN:

PRINT restock header

CALL restock_product(products) FUNCTION FROM 'operation' MODULE

STORE returned result IN 'result'

IF result IS NOT None THEN:

UNPACK result INTO:

- restocked_items

- total_restock_cost

PROMPT user to confirm saving restocking bill to text file (y/n)

IF user_input IS 'y' THEN:

CALL save_restocking_bill(restocked_items, total_restock_cost) FUNCTION FROM 'write' MODULE

ELSE IF option == 3 THEN:

PRINT exit message

CALL write_products(products) FUNCTION FROM 'write' MODULE TO SAVE updated product list

```
        SET main_loop = FALSE
    ELSE:
        PRINT invalid option message (not 1-3)
END WHILE
END PROGRAM
```

4.2 Operations module

```
IMPORT ‘datetime’
IMPORT ‘save_sold_bill’,’save_restocking_bill’ and ‘write_products FROM ‘write’ MODULE

FUNCTION display_products(products):
  PRINT header for product table (ID, Name, Brand, Quantity, Price, Country)
  FOR each key-value pair in products dictionary:
      DISPLAY product ID (key)
      EXTRACT cost price from index 3 of value list
      CALCULATE selling price = cost_price *2
      FOR each index in product details list:
          IF index 3 (cost price):
              DISPLAY calculated selling price
          ELSE:
              DISPLAY other attributes (name, brand, quantity, country)
  PRINT a line in footer
END FUNCTION

FUNCTION sell_product(products):
   LOOP UNTIL valid customer name is entered:
      PROMPT user to input name
      IF name is empty OR is only digits:
         DISPLAY appropriate error message
      ELSE:
         BREAK loop
   LOOP UNTIL valid phone number is entered:
      PROMPT user to input phone number
      IF phone number is all digits AND has 10 digits:
         BREAK loop
      ELSE:
         DISPLAY error message
```

```
INITIALIZE:
    sold_items as empty list
    total = 0
    shipping_cost = 0
    sell_loop = True

WHILE sell_loop is True:
    CALL display_products(products)

    TRY:
        PROMPT user to input product ID
        WHILE ID is invalid (out of range or non-positive):
            DISPLAY error
            PROMPT again

        PROMPT for product quantity

    EXCEPT ValueError:
        DISPLAY error
        CONTINUE loop

    CALCULATE:
        free_item = item_quantity // 3
        stock_deduct = item_quantity + free_item
        available_quantity = products[item_id][2]

    WHILE quantity is invalid OR stock is insufficient:
        DISPLAY appropriate error
        PROMPT again for quantity
        RECALCULATE free_item and stock_deduct
```

```
        IF item_quantity >= 3:
            DISPLAY message about free items offer

        UPDATE products[item_id][2] by subtracting stock_deduct
        CALL write_products(products) to save updated stock

        CALCULATE:
            selling_price = cost_price * 2
            total_price = selling_price * item_quantity
            total += total_price

        APPEND [name, quantity, unit price, total] TO sold_items list

        ASK if user wants to sell another product
        IF not 'y', SET sell_loop = False

    IF total > 10000:
        SET shipping_cost = 0
    ELSE:
        SET shipping_cost = 1200
    CALCULATE grand_total = total + shipping_cost

    DISPLAY bill header
    DISPLAY current date and time
    DISPLAY customer details
    DISPLAY each sold product with quantity, unit price, and total
    DISPLAY subtotal, shipping, and grand total

    RETURN name, phone number, sold_items list, total, shipping_cost,
    grand_total
END FUNCTION
```

```
FUNCTION restock_product(products):

    INITIALIZE:
        restocked_items = empty list
        total_restock_cost = 0
        restock_loop = True

    WHILE restock_loop is True:
        CALL display_products(products)

        TRY:
            PROMPT user for restock_id
            WHILE ID is invalid:
                DISPLAY error and PROMPT again

            PROMPT user for restock_quantity
            WHILE quantity <= 0:
                DISPLAY error and PROMPT again

        EXCEPT ValueError:
            DISPLAY input error
            CONTINUE loop

        UPDATE product quantity in products dictionary
        CALL write_products(products) to save updated data

        CALCULATE:
            cost_price = products[restock_id][3]
            total_price = cost_price * restock_quantity
            total_restock_cost += total_price
```

```
        APPEND [product name, quantity, unit cost, total cost] to restocked_items

        DISPLAY confirmation message and updated product list

        ASK if user wants to restock another item
        IF not 'y', SET restock_loop = False

    DISPLAY restocking bill:
        PRINT current date and time
        DISPLAY restocked product details
        DISPLAY total restocking cost

    RETURN restocked_items, total_restock_cost

END FUNCTION
```

4.3 Write Module

```
IMPORT ‘datetime’
FUNCTION write_products(products):
    OPEN "product_stock.txt" file in write mode
    WRITE header line with column titles (ID, Name, Brand, Quantity, Price,
    Country)
    WRITE a separator line using '=' repeated 90 times

    FOR each product in products dictionary:
        EXTRACT cost price from index 3 of the value list
        CALCULATE selling price = cost_price * 2
        FORMAT line containing: ID, Name, Brand, Quantity, Selling Price, Country
        WRITE the formatted line to the file

    CLOSE file automatically using 'with' block

END FUNCTION

FUNCTION save_sold_bill(customer_name, phone, sold_items, total,
shipping_cost, grand_total):

    GET current datetime using datetime.now()
```

FORMAT datetime as string (YYYY-MM-DD HH:MM:SS) manually using string concatenation and zfill

GENERATE filename using customer_name and timestamp:
- Remove spaces from name
- Append date and time digits to create unique filename

TRY:
OPEN the generated filename in write mode

WRITE bill header
WRITE formatted current datetime
WRITE customer name and phone number

WRITE table headers for products (Product, Quantity, Unit Price, Total Price)
FOR each item in sold_items:
WRITE product name, quantity, unit price, and total price in a formatted row

WRITE subtotal, shipping cost, and grand total

CLOSE file (automatic in 'with' block)
PRINT confirmation message with filename

EXCEPT any error during file operation:
PRINT error message with exception details

END FUNCTION

FUNCTION save_restocking_bill(restocked_items, total_restock_cost):

GET current datetime using datetime.now()

FORMAT datetime as string (YYYY-MM-DD HH:MM:SS) using string concatenation and zfill

GENERATE unique filename using date and time information

TRY:

OPEN the generated filename in write mode

WRITE bill header for restocking

WRITE formatted current datetime

WRITE table headers (Product, Quantity, Unit Cost, Total Cost)

FOR each item in restocked_items:

WRITE item details (name, quantity, unit cost, total cost) in tabular format

WRITE total restocking cost

CLOSE file (automatic in 'with' block)

PRINT confirmation message with filename

EXCEPT any error during file writing:

PRINT error message with exception details

END FUNCTION

4.4 Read Module

```
FUNCTION read_products():

    TRY:
        OPEN "Product.txt" file in read mode

        CREATE an empty dictionary product_dict

        READ all lines from the file into product_lines list

        INITIALIZE item_id as 1 to represent the first product ID

        FOR each line in product_lines:
            REMOVE newline character from the line using line.replace("\n", "")
            SPLIT the line by comma into a list of product details
            INSERT the product details list into product_dict with item_id as key
            INCREMENT item_id by 1 to assign the next ID to the next product

        PRINT the product_dict dictionary to show the loaded products

        RETURN product_dict

    EXCEPT FileNotFoundError:
        PRINT error message "Product.txt file not found!" and exit the program

    EXCEPT any other Exception:
        PRINT error message with exception details, and exit the program

END FUNCTION
```

5. Data Structures

This section highlights the data structures used in the development of the program. Each structure plays a major role in organizing, storing and processing the data.

A. Data Type

i. Int

Int is a whole number that does not have decimal.

For example, x = 10.

ii. Float

Float is a number with decimal points.

For example, x = 10.12

iii. Complex

Complex number with a real and imaginary part

For example, z = 2+3j

iv. Boolean

It is used to give true or false input

For example, is_value = true

v. Str

It is a datatype where string is formed with sequence of characters

For example, var = “python”

B. Non-primitive Datatype

I. List

List is used to store the data in a list after it is read and structured properly from the txt file. It allows us to access product attributes for displaying or processing.

For example:

[‘Safe SunUV’ , ‘Lotus’ , ‘400’ , ‘850’ , ‘India’]

The above attributes can be accessed easily like product[0] gets the name , product[1] gets the company etc.

II. Dictionary

Dictionary is used to store products record where each key is unique and the corresponding value is a list containing product details like name, company, price etc. This allows efficient access and organization of multiple product entry.

For example:

d = { 1: [‘Safe SunUV’ , ‘Lotus’ , ‘400’ , ‘850’ , ‘India’],

2: [‘Aveeno Protect’ , ‘Aveeno’ , ‘300’ , ‘900’ , ‘USA’]

}

III. String

The structure is used to format each line read from the text file which is initially a string containing product data in a raw form. It is used to remove newline character and splitting fields using comma.

For example: .replace(“\n”,””) is used to remove the newline character and .split(“ , “) is used to separate each field using comma.

For example:

The raw data in txt file is like this: Safe SunUV, Lotus, 400, 850, India

After using string structure It is stored like this: “Safe SunUV,Lotus,400,850 ,India”

IV. Tuple

A tuple is very similar to list but it is immutable, which means that the content jn tuple cannot be changed after its created. It is ordered and allows duplicates. It is mostly used to store fixed sets of values like coordinates or product attributed etc.

For example:

Product = (“Aveeno Protect”, “Aveeno”, “300”, “900”, “USA)

V. Set

A set is an unordered collection of unique items which does not allow duplicates. It is used for operations like performing set operations that is union, intersection, removing duplicates etc.

For example:

Brands = {“Lotus”, “Aveeno”, “Neutrogena”}

6. Program

Figure 4: Execution of Program step 1

```
Enter an option to proceed: 1


-------------------------------------------------------------------------------------------------------
Enter your details to generate bill
-------------------------------------------------------------------------------------------------------


Enter the customer's name: Jamie
Enter the phone number: 0899332211
-------------------------------------------------------------------------------------------------------


*******************************************************************************************************
id       Name                   Brand          Quantity        Price           Country
*******************************************************************************************************
1       Safe SunUV              Lotus           80              800             India
2       Aveeno Protect          Aveeno          65              600             USA
3       Sun Protect             Nivea           50              1200            Germany
*******************************************************************************************************


Please provide the Id of the product to sell: 1



Please enter the quantity of product to sell: 25


********** Dear Jamie, as per the buy 3 get 1 free offer going on, you have received 8 Safe SunUV for free. **********


47      Do you want to sell another product? (y/n): y


*******************************************************************************************************
id       Name                   Brand          Quantity        Price           Country
*******************************************************************************************************
1       Safe SunUV              Lotus          47              800             India
2       Aveeno Protect          Aveeno          65              600             USA
3       Sun Protect             Nivea           50              1200            Germany
*******************************************************************************************************


Please provide the Id of the product to sell: 2



Please enter the quantity of product to sell: 45


********** Dear Jamie, as per the buy 3 get 1 free offer going on, you have received 15 Aveeno Protect for free. **********


5       Do you want to sell another product? (y/n): n
```

Figure 5: Execution of Program step 2

```
5       Do you want to sell another product? (y/n): n


=============================================================================================================
                                        WeCare Wholesale Bill
=============================================================================================================
Customer Name:    Jamie
Phone Number:     0899332211
-------------------------------------------------------------------------------------------------------------
Product          Quantity        Unit Price      Total Price
-------------------------------------------------------------------------------------------------------------
Safe SunUV       25               800             20000
Aveeno Protect   45               600             27000
-------------------------------------------------------------------------------------------------------------
Subtotal:        47000
Shipping Cost:   0
Grand Total:     47000
=============================================================================================================


-------------------------------------------------------------------------------------------------------------
Displaying your options below
-------------------------------------------------------------------------------------------------------------
Option 1 (sell the product)
Option 2 (Buy from Manufacturer)
Option 3 (Exit the system)
-------------------------------------------------------------------------------------------------------------
```

Figure 6: Execution of Program step 3

```
Enter an option to proceed: 2


-------------------------------------------------------------------------------------------------------------
Restock from Manufacturer
-------------------------------------------------------------------------------------------------------------


*************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
*************************************************************************************************************
1       Safe SunUV               Lotus          47              800              India
2       Aveeno Protect           Aveeno         5               600              USA
3       Sun Protect              Nivea           50             1200             Germany
*************************************************************************************************************


Please provide the Id of the product to restock: 1


Please enter the quantity of product to add: 80


Stock successfully updated!

*************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
*************************************************************************************************************
1       Safe SunUV               Lotus          127             800              India
2       Aveeno Protect           Aveeno         5               600              USA
3       Sun Protect              Nivea           50             1200             Germany
*************************************************************************************************************


Do you want to restock another product? (y/n): n


-------------------------------------------------------------------------------------------------------------
Displaying your options below
-------------------------------------------------------------------------------------------------------------
Option 1 (sell the product)
Option 2 (Buy from Manufacturer)
Option 3 (Exit the system)
-------------------------------------------------------------------------------------------------------------


Enter an option to proceed:
```

Figure 7: Execution of Program step 4

```

-------------------------------------------------------------------------------------------------------------
Displaying your options below
-------------------------------------------------------------------------------------------------------------
Option 1 (sell the product)
Option 2 (Buy from Manufacturer)
Option 3 (Exit the system)
-------------------------------------------------------------------------------------------------------------


Enter an option to proceed: 3


Exiting the system. Thank you!
>>>
```

Figure 8: Execution of Program step 5

7. Testing

7.1 Test 1

Objective	To show implementation of try, except and print a message for invalid input
Action	• Implement a try-except block to handle an exception for user input validation • Enter an invalid input
Expected result	• The exception handling for user input validation should be created • An error message should display

Actual Result	• The exception handling for user input validation was created • An error message was displayed when invalid input was given
Conclusion	Exception handling using try, except was conducted and test was passed.

Table 1: Test 1

```
#Getting valid product Id from the customer
        try:
            item_id = int(input("Please provide the Id of the product to sell: "))
            print("\n")

            #Verifying product Id
            while item_id <= 0 or item_id > len(products):
                print("\n**********************Please provide a valid Id!!!**********************\n")
                item_id = int(input("Please Provide the Id of the product to sell: "))

            #Taking Quantity
            item_quantity = int(input("Please enter the quantity of product to sell: "))
        except ValueError:
            print("Invalid input. Please enter valid numbers.\n")
            continue
```

Figure 9: Implementation of try, except

```
Enter your details to generate bill
-----------------------------------------------------------------------------------------------------
Enter the customer's name: Aasanna
Enter the phone number: 9841251533
*****************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
*****************************************************************************************************
1       Safe SunUV               Lotus          47              800              India
2       Aveeno Protect           Aveeno          65             600              USA
3       Sun Protect              Nivea           45             1200             Germany
*****************************************************************************************************


Please provide the Id of the product to sell: -123



**********************Please provide a valid Id!!!**********************

Please Provide the Id of the product to sell:
```

Figure 10: Message display for invalid input

7.2 Test 2

Objective	To test validation for purchase and sale of product
Action	• Enter a negative value while selection option • Enter a non-existence Id while choosing products
Expected result	• An error message “Please provide a valid Id!!!” should display for both.
Actual Result	• An error message “Please provide a valid Id!!!” was displayed
Conclusion	Input validation for invalid inputs was conducted, and test was passed

Table 2: Test 2

```
Enter an option to proceed: -87


Invalid option. Please select between 1-3.

-----------------------------------------------------------------------------------------------------------
Displaying your options below
-----------------------------------------------------------------------------------------------------------
Option 1 (Sell the product)
Option 2 (Buy from Manufacturer)
Option 3 (Exit the system)
-----------------------------------------------------------------------------------------------------------


Enter an option to proceed:
```

Figure 11: Error Message for entering negative option

```
Enter an option to proceed: 1


-------------------------------------------------------------------------------------------------------------
Enter your details to generate bill
-------------------------------------------------------------------------------------------------------------
Enter the customer's name: joe
Enter the phone number: 9810224590
*************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
*************************************************************************************************************
1       Safe SunUV               Lotus           47              800             India
2       Aveeno Protect           Aveeno           65             600             USA
3       Sun Protect              Nivea            45             1200            Germany
*************************************************************************************************************


Please provide the Id of the product to sell: 9


********************Please provide a valid Id!!!********************

Please Provide the Id of the product to sell:
```

Figure 12: Error message for entering non-existent Id while selling

```
Enter an option to proceed: 2


-------------------------------------------------------------------------------------------------------------
Restock from Manufacturer
-------------------------------------------------------------------------------------------------------------
*************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
*************************************************************************************************************
1       Safe SunUV               Lotus           47              800             India
2       Aveeno Protect           Aveeno           65             600             USA
3       Sun Protect              Nivea            45             1200            Germany
*************************************************************************************************************


Please provide the Id of the product to restock: 12
Please provide a valid Id!!!

Please provide the Id of the product to restock:
```

Figure 13: Error message for entering non-existent Id while buying

7.3 Test 3

Objective	To show file generation of purchase process of multiple products
Action	• Show the complete purchase process • Show an output in the shell • Show the purchased products detail in a text file
Excepted Result	• The whole process is conducted

	• An output should display in the shell • A text file of purchased products detail should be created
Actual Result	• The whole process was conducted • An output was displayed in the shell • A text file was created showing the details
Conclusion	The test was conducted successfully

Table 3: Test 3

```
--------------------------------------------------------------------------------------------------------
Restock from Manufacturer
--------------------------------------------------------------------------------------------------------
********************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
********************************************************************************************************
1       Safe SunUV               Lotus          14              800              India
2       Aveeno Protect           Aveeno         41              600              USA
3       Sun Protect              Nivea           45             1200             Germany
********************************************************************************************************


Please provide the Id of the product to restock: 1
Please enter the quantity of product to add: 250
Stock successfully updated!

********************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
********************************************************************************************************
1       Safe SunUV               Lotus          264             800              India
2       Aveeno Protect           Aveeno         41              600              USA
3       Sun Protect              Nivea           45             1200             Germany
********************************************************************************************************


Do you want to restock another product? (y/n): y
********************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
********************************************************************************************************
1       Safe SunUV               Lotus          264             800              India
2       Aveeno Protect           Aveeno         41              600              USA
3       Sun Protect              Nivea           45             1200             Germany
********************************************************************************************************


Please provide the Id of the product to restock: 2
Please enter the quantity of product to add: 150
Stock successfully updated!

********************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
********************************************************************************************************
1       Safe SunUV               Lotus          264             800              India
2       Aveeno Protect           Aveeno         191             600              USA
3       Sun Protect              Nivea           45             1200             Germany
********************************************************************************************************


Do you want to restock another product? (y/n): n
========================================================================================================
                                        WeCare Restocking Bill
========================================================================================================
Date & Time:     2025-05-14 09:11:12.951841
--------------------------------------------------------------------------------------------------------
Product         Quantity        Unit Cost       Total Cost
--------------------------------------------------------------------------------------------------------
Safe SunUV       250             400             100000
Aveeno Protect   150             300             45000
--------------------------------------------------------------------------------------------------------
Total Restocking Cost:   145000
========================================================================================================


Do you want to save this bill to a text file? (y/n): y
Restocking bill saved successfully as WeCare Bill202551491114.txt
```

Figure 14: Purchase Process of multiple products displayed in a shell

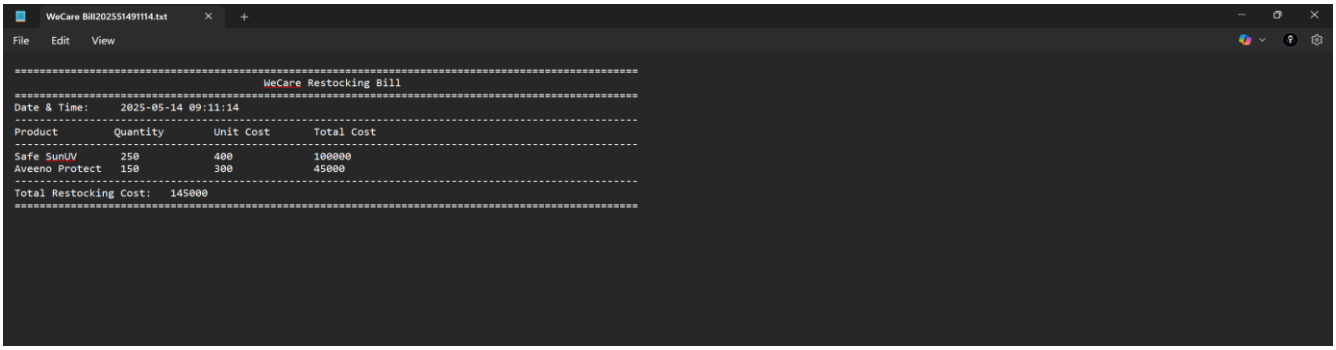

Figure 15: A text file with details of purchased products

7.4 Test 4

Objective	To show file generation of sales process of multiple products
Action	• Show the complete sales process • Show an output in the shell • Show the sold products detail in a text file
Expected Result	• The whole process is conducted • An output should display in the shell • A text file of sold products detail should be created
Actual Result	• The whole process was conducted • An output was displayed in the shell • A text file was created showing the details
Conclusion	The test was conducted successfully

Table 4: Test 4

```
Please provide the Id of the product to sell: 1


Please enter the quantity of product to sell: 25
********** Dear Joe, as per the buy 3 get 1 free offer going on, you have received 8 Safe SunUV for free. **********
************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
************************************************************************************************************
1        Safe SunUV              Lotus          231             800              India
2        Aveeno Protect          Aveeno         191             600              USA
3        Sun Protect             Nivea           45             1200             Germany
************************************************************************************************************


Do you want to sell another product? (y/n): y
************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
************************************************************************************************************
1        Safe SunUV              Lotus          231             800              India
2        Aveeno Protect          Aveeno         191             600              USA
3        Sun Protect             Nivea           45             1200             Germany
************************************************************************************************************


Please provide the Id of the product to sell: 3


Please enter the quantity of product to sell: 10
********** Dear Joe, as per the buy 3 get 1 free offer going on, you have received 3 Sun Protect for free. **********
************************************************************************************************************
id       Name                    Brand          Quantity        Price           Country
************************************************************************************************************
1        Safe SunUV              Lotus          231             800              India
2        Aveeno Protect          Aveeno         191             600              USA
3        Sun Protect             Nivea          32              1200             Germany
************************************************************************************************************


Do you want to sell another product? (y/n): n
============================================================================================================
                                        WeCare Wholesale Bill
============================================================================================================
Date & Time:      2025-05-14 09:19:26.597509
Customer Name:    Joe
Phone Number:     9810224590
------------------------------------------------------------------------------------------------------------
Product          Quantity        Unit Price      Total Price
------------------------------------------------------------------------------------------------------------
Safe SunUV       25              800             20000
Sun Protect      10              1200            12000
------------------------------------------------------------------------------------------------------------
Subtotal:        32000
Shipping Cost:   0
Grand Total:     32000
============================================================================================================


Do you want to save this bill to a text file? (y/n): y
Bill saved successfully as WeCare_Bill_Joe202551491929.txt
```

Figure 16: Sales Process of multiple products displayed in a shell

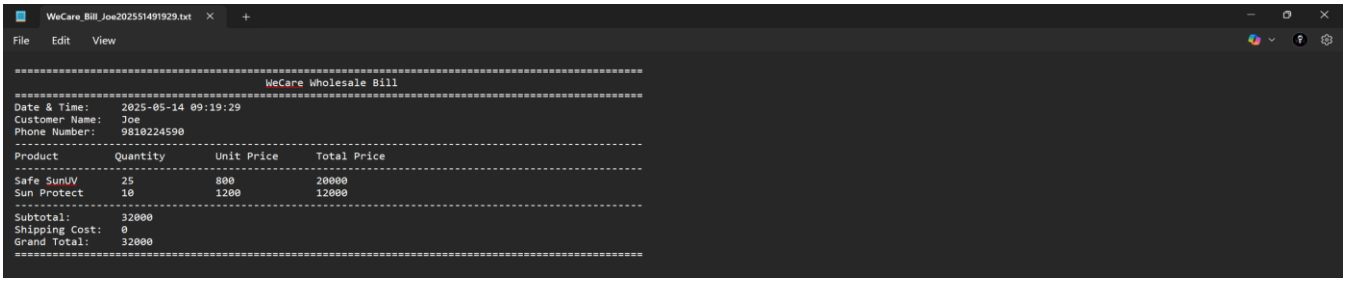

Figure 17: A text file with details of sold products

7.5 Test 5

Objective	To show the update in stock of the products
Action	• Show the quantity being added while purchasing the product in .txt file • Show the quantity being deducted while selling product in .txt file
Expected Result	• The stock should be added and updated stock should display in .txt file • The stock was deducted and updated stock was displayed in .txt file
Actual Result	• The stock should be added and updated stock should display in .txt file • The stock was deducted and updated stock was displayed in .txt file
Conclusion	The updated stock was displayed for both test and test were passed.

Table 5: Test 5

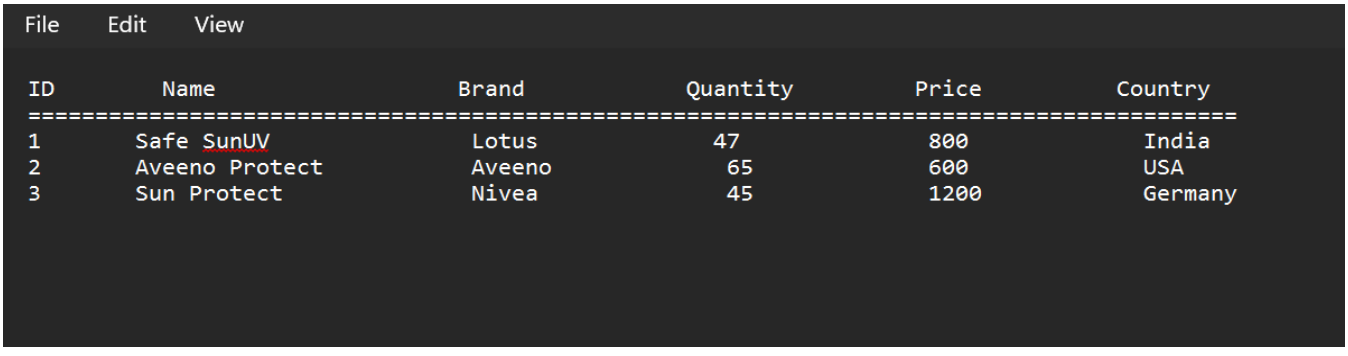

Figure 18: Initial stock

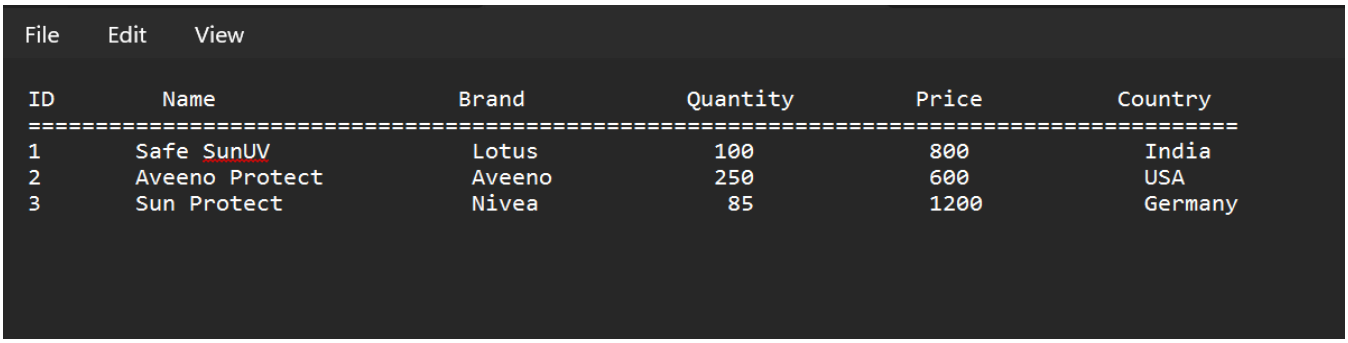

Figure 19: Stock after purchasing

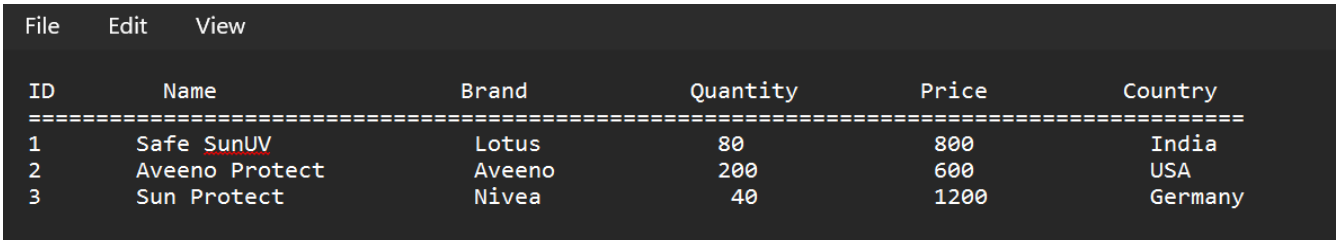

Figure 20: Stock after selling products

8. Conclusion

The course work of the module “Fundamentals of Computing” effectively illustrates the use of essential programming principles to develop a working inventory management system for weCare Wholesale. By utilizing the pythons features like data structures, and error handling, the project was made to meet the goal and requirements of reading, processing and presenting product information in a orderly fashion.

The project emphasizes the significance of structured programming, incorporating distinct modularization into read, write, and operations sections, which guarantees maintainability and scalability. Testing verified the system's strength, managing edge cases such as invalid entries and stock validation efficiently. The creation of comprehensive invoices and immediate inventory updates further highlights the project's relevance in practical scenarios.

In general, this project acts as a basic step toward more advanced systems, strengthening crucial programming abilities while offering a practical solution for managing inventory. Potential improvements might involve graphical interfaces, database integration, or sophisticated analytics to enhance its functionality further more.

9. Appendix

#Main.py#

```
from read import read_products
from write import save_sold_bill, save_restocking_bill
from operation import sell_product, restock_product, display_products

print("\n" * 2)
print("\t\t\t\t\t\tWeCare Wholesale\n")
print("\t\t\t\t   Kamalpokhari, Kathmandu | Ph no: 01445833\n")
print("-" * 112)
print("\t\t\tWelcome to the system! Hope you have a great time")
print("-" * 112)
print("\n")

products = read_products()

#Looping options for admin
main_loop = True
while main_loop:
    print("-" * 100)
    print("Displaying your options below")
    print("-" * 100)
    print("Option 1 (Sell the product)")
    print("Option 2 (Buy from Manufacturer)")
    print("Option 3 (Exit the system)")
    print("-" * 100)
    print("\n")

    #Taking admin's choice
    try:
        option = int(input("Enter an option to proceed: "))
        print("\n")
    except ValueError:
        print("Invalid input. Please enter a number (1-3).\n")
        continue

#For option 1
    if option == 1:
        print("-" * 100)
        print("Enter your details to generate bill")
        print("-" * 100)
```

```
        result=sell_product(products)

        if result is not None:
            name, ph_no, sold_items, total, shipping_cost, grand_total = result
            save = input("Do you want to save this bill to a text file? (y/n): ").lower()
            if save == 'y':
                save_sold_bill(name, ph_no, sold_items, total, shipping_cost,
grand_total)

#For option 2
    elif option == 2:
        print("-" * 100)
        print("Restock from Manufacturer")
        print("-" * 100)
        result = restock_product(products)

        if result is not None:
            restocked_items, total_restock_cost = result
            save = input("Do you want to save this bill to a text file? (y/n): ")
            if save == 'y':
                save_restocking_bill(restocked_items, total_restock_cost)

#For option 3
    elif option == 3:
        print("Exiting the system. Thank you!")
        write_products(products)
        main_loop = False
    else:
        print("Invalid option. Please select between 1-3.\n")
```

```
#Operations.py#

import datetime
from write import save_sold_bill, save_restocking_bill, write_products
#Displaying products that are available to buy
def display_products(products):
    print("*" * 100)
    print("id  \t Name\t\t\t Brand\t\tQuantity \tPrice \t\tCountry")
    print("*" * 100)
    for key, value in products.items():
        print(key, end="\t")
        cost_price = int(value[3])
        selling_price = cost_price * 2
        for i in range(len(value)):
            if i == 3:
                #Displaying selling price of that is double of cost price
                print(selling_price, end="\t\t")
            else:
                print(value[i], end="\t\t")
        print()
    print("*" * 100)
    print("\n")

def sell_product(products):
    while True:

        name = input("Enter the customer's name: ")
        if name == "":
            print("\n***** Name cannot be empty! Please enter a valid name. *****\n")
        elif name.isdigit():
            print("\n***** Name cannot be a integer! Please enter a valid name. *****\n")

        else:
            break

    while True:
        ph_no = input("Enter the phone number: ")
        if ph_no.isdigit() and len(ph_no) == 10:
            break
        else:
            print("\n*****Invalid phone number, Please enter a 10 digit number!*****\n")

    sold_items = []
    total = 0
```

```
        shipping_cost = 0
        sell_loop = True

        while sell_loop:
            display_products(products)

    #Getting valid product Id from the customer
            try:
                item_id = int(input("Please provide the Id of the product to sell: "))
                print("\n")

                #Verifying product Id
                while item_id <= 0 or item_id > len(products):
                    print("\n********************Please provide a valid Id!!!********************\n")
                    item_id = int(input("Please Provide the Id of the product to sell: "))

                #Taking Quantity
                item_quantity = int(input("Please enter the quantity of product to sell: "))
            except ValueError:
                print("Invalid input. Please enter valid numbers.\n")
                continue

    #Free items (Buy 3 get 1 free)
            selected_product_quantity = products[item_id][2]
            free_item = item_quantity // 3
            stock_deduct = item_quantity + free_item

    #Validating available stock
            while item_quantity <= 0 or stock_deduct > int(selected_product_quantity):
                if item_quantity <= 0:
                    print("The given input is not valid, Please enter a valid item quantity")
                elif stock_deduct > int(selected_product_quantity):
                    print("The stock for selected item is not available. Please look into
    available stock!\n")

                try:
                    item_quantity = int(input("Please enter the quantity of product to buy: "))
                    free_item = item_quantity // 3
                    stock_deduct = item_quantity + free_item
                except ValueError:
                    print("Invalid quantity.\n")
                    continue

    #Announcing about the free items scheme
            if item_quantity >= 3:
```

```
            print("*" * 10 + " Dear " + name + ", as per the buy 3 get 1 free offer going
on, you have received " +str(free_item) + " " + products[item_id][0] + " for free. " +
"*" * 10)
        products[item_id][2] = str(int(products[item_id][2]) - stock_deduct)
        display_products(products)
        write_products(products)

        cost_price = int(products[item_id][3])
        selling_price = cost_price * 2
        total_price = selling_price * item_quantity
        total += total_price

#Appending the item sold for billing
        sold_items.append([products[item_id][0], item_quantity, selling_price,
total_price])
        more = input("Do you want to sell another product? (y/n): ").lower()
        if more != "y":
            sell_loop = False

    if total > 10000:
        shipping_cost = 0
    else:
        shipping_cost = 1200

    grand_total = total + shipping_cost

#Generating the bill
    print("=" * 100)
    print("\t\t\t\t\tWeCare Wholesale Bill")
    print("=" * 100)
    now = datetime.datetime.now()
    print("Date & Time:\t", now)#Adding the current date and time of the bill
generated
    print("Customer Name:\t", name)
    print("Phone Number:\t", ph_no)
    print("-" * 100)
    print("Product\t\tQuantity\tUnit Price\tTotal Price")
    print("-" * 100)
    for item in sold_items:
        print(item[0], "\t", item[1], "\t\t", item[2], "\t\t", item[3])
    print("-" * 100)
    print("Subtotal:\t", total)
    print("Shipping Cost:\t", shipping_cost)
    print("Grand Total:\t", grand_total)
    print("=" * 100)
    print("\n")
```

```
    return name, ph_no, sold_items, total, shipping_cost, grand_total


def restock_product(products):

    restock_loop = True
    restocked_items = []
    total_restock_cost = 0
    while restock_loop:
        display_products(products)#Display product before restocking
        try:
            restock_id = int(input("Please provide the Id of the product to restock: "))
            while restock_id <= 0 or restock_id > len(products):
                print("Please provide a valid Id!!!\n")
                restock_id = int(input("Please provide the Id of the product to restock: "))
            restock_quantity = int(input("Please enter the quantity of product to add: "))
            while restock_quantity <= 0:
                print("Please enter a valid quantity greater than 0!!!\n")
                restock_quantity = int(input("Please enter the quantity of product to add:
"))
        except ValueError:
            print("Invalid input. Please enter valid numbers.\n")
            continue

        products[restock_id][2] = str(int(products[restock_id][2]) + restock_quantity)
        write_products(products)
        cost_price = int(products[restock_id][3])
        total_price = cost_price * restock_quantity
        total_restock_cost += total_price

        restocked_items.append([products[restock_id][0], restock_quantity, cost_price,
total_price])

        print("Stock successfully updated!\n")
        display_products(products)

        more = input ("Do you want to restock another product? (y/n): ").lower()
        if more != "y":
            restock_loop = False


    # Generate the restocking bill
    print("=" * 100)
    print("\t\t\t\t\tWeCare Restocking Bill")
    print("=" * 100)
```

```
    now = datetime.datetime.now()
    print("Date & Time:\t", now)
    print("-" * 100)
    print("Product\t\tQuantity\tUnit Cost\tTotal Cost")
    print("-" * 100)
    for item in restocked_items:
        print(item[0], "\t", item[1], "\t\t", item[2], "\t\t", item[3])
    print("-" * 100)
    print("Total Restocking Cost:\t", total_restock_cost)
    print("=" * 100)
    print("\n")

    return restocked_items, total_restock_cost
```

```
#Write.py#

import datetime
def write_products(products):
    with open("product_stock.txt", "w") as file:
        file.write("ID\t  Name\t\t\tBrand\t\t Quantity\t  Price\t\t Country\n")
        file.write("=" * 90 + "\n")
        for key, value in products.items():
            cost_price = int(value[3])
            selling_price = cost_price * 2
            line = str(key) + "\t" + value[0] + "\t\t" + value[1] + "\t\t   " + value[2] + "\t\t   " +
str(selling_price) + "\t\t  " + value[4] + "\n"
            file.write(line)


def save_sold_bill(customer_name, phone, sold_items, total, shipping_cost,
grand_total):
    now = datetime.datetime.now()


    now_string = str(now.year) + "-" + str(now.month).zfill(2) + "-" +
str(now.day).zfill(2) + " " + str(now.hour).zfill(2) + ":" + str(now.minute).zfill(2) + ":" +
str(now.second).zfill(2)

    filename = "WeCare_Bill_"+customer_name.replace(" ","_") + str(now.year) +
str(now.month) + str(now.day) + str(now.hour) + str(now.minute) + str(now.second)
+ ".txt"

    try:
        with open(filename, "w") as file:
            file.write("=" * 100 + "\n")
            file.write("\t\t\t\t\tWeCare Wholesale Bill\n")
            file.write("=" * 100 + "\n")
            file.write("Date & Time:\t " + now_string + "\n")
            file.write("Customer Name:\t " + customer_name + "\n")
            file.write("Phone Number:\t " + phone + "\n")
            file.write("-" * 100 + "\n")
            file.write("Product\t\tQuantity\tUnit Price\tTotal Price\n")
            file.write("-" * 100 + "\n")
            for item in sold_items:
                file.write(item[0] + "\t " + str(item[1]) + "\t\t" + str(item[2]) + "\t\t" +
str(item[3]) + "\n")
            file.write("-" * 100 + "\n")
```

```
            file.write("Subtotal:\t " + str(total) + "\n")
            file.write("Shipping Cost:\t " + str(shipping_cost) + "\n")
            file.write("Grand Total:\t " + str(grand_total) + "\n")
            file.write("=" * 100 + "\n")
            file.write("\n")
        print("Bill saved successfully as " + filename)
    except Exception as e:
        print("\n*****Error saving bill:", str(e)+"*****")


def save_restocking_bill(restocked_items, total_restock_cost):
    now = datetime.datetime.now()


    now_string = str(now.year) + "-" + str(now.month).zfill(2) + "-" +
str(now.day).zfill(2) + " " + str(now.hour).zfill(2) + ":" + str(now.minute).zfill(2) + ":" +
str(now.second).zfill(2)

    filename = "WeCare Bill" + str(now.year) + str(now.month) + str(now.day) +
str(now.hour) + str(now.minute) + str(now.second) + ".txt"

    try:
        with open(filename, "w") as file:
            file.write("=" * 100 + "\n")
            file.write("\t\t\t\t\tWeCare Restocking Bill\n")
            file.write("=" * 100 + "\n")
            file.write("Date & Time:\t " + now_string + "\n")
            file.write("-" * 100 + "\n")
            file.write("Product\t\tQuantity\tUnit Cost\tTotal Cost\n")
            file.write("-" * 100 + "\n")
            for item in restocked_items:
                file.write(item[0] + "\t " + str(item[1]) + "\t\t" + str(item[2]) + "\t\t" +
str(item[3]) + "\n")
            file.write("-" * 100 + "\n")
            file.write("Total Restocking Cost:\t " + str(total_restock_cost) + "\n")
            file.write("=" * 100 + "\n")
            file.write("\n")
        print("Restocking bill saved successfully as " + filename)
    except Exception as e:
        print("*****Error saving restocking bill:", str(e)+"*****")
```

```
#Read.py#

def read_products():
    try:
        file = open("Product.txt", "r")
        product_dict = {}
        product_lines = file.readlines()
        item_id = 1
        for line in product_lines:
            line = line.replace("\n","").split(",")
            product_dict[item_id] = line # value insertion
            item_id += 1
        #Display loaded products from the dictionary
        print(product_dict)
        print("\n")
        return product_dict
    except FileNotFoundError:
        print("Error: Product.txt file not found!")
        exit()
    except Exception as error:
        print("An unexpected error occurred while reading the file:", str(error))
        exit()
```